

UE Projet informatique - L1 S2

Projet : RobotMaster

Année académique 2025/2026

Contents

1	Consignes et organisation du projet	2
2	Le thème du projet : RobotMaster	4
2.1	Règles du jeu	4
3	Implémentations - Informations générales	5
3.1	Les variables principales	5
3.2	Les types utilisés	5
3.3	Les tests	6
4	Implémentations - Partie Guidée	7
4.a	Fichier a_plateau.py	7
4.b	Fichier b_gestionCartes.py	8
4.c	Fichier c_joueuses.py	9
4.d	Fichier d_score.py	10
4.e	Fichier e_jeu.py	10
5	Implémentations - Stratégie et IA	11
5.c	Fichier c_joueuses_IA.py	11
5.e	Fichier e_jeu_IA.py	11
5.f	Fichier f_fonctions_additionnelles.py	11
5.g	Fichier g_greedy.py	11
5.h	Fichier h_agressif.py	12
5.i	Fichier i_meilleurIA.py (extension battez les profs)	12
6	Partie Extensions	13
6.1	Propositions	13
6.2	Vos extensions	13
A	Annexe – Configuration du dépôt GitLab	14
A.1	Bien démarrer	14
A.2	Créer un projet GitLab	14
A.3	Premier commit	15
A.4	Pour la suite	15
A.5	Mémo git	15

1 Consignes et organisation du projet

Le projet est à faire en binôme et comprend trois volets techniques :

1. un programme à réaliser en Python, contenant toutes les fonctions demandées dans les parties guidée et IA, ainsi qu'une quantité satisfaisante d'extensions (voire extensions pour plus de détails).
 - Vos dossiers `partie_guidée` et `IA` contiennent les fichiers avec toutes les fonctions demandées.
Ces fonctions doivent respecter les spécifications demandées de manière exacte.
 - Les algorithmes et l'implémentation sont bien choisis (pas de solutions inutilement compliquées, même si le résultat est le bon).
 - La quantité et la qualité des extensions réalisées.
 - Votre code est propre, bien lisible, élégant, correctement indenté (présentation, noms de variable explicites, commentaires utiles, ...)
 - **Votre dossier extensions doit contenir une fonction main qui correspond au programme principal qui est exécuté.** Ce programme principal permet, au lancement, d'avoir un bon aperçu des fonctionnalités implémentées et est agréable à utiliser (faites des choix pertinents). Votre enseignant doit pouvoir lancer directement le code et admirer le résultat, sans erreurs bien sûr, et avec la possibilité de pouvoir modifier les entrées du programme très simplement.
2. un rapport à rédiger en $\text{\LaTeX}$, **Max 5 pages**, qui devra :
 - contenir les noms des membres du groupe ainsi que le numéro du binôme choisi sur Moodle
 - **contenir un lien hypertexte vers le dépôt gitlab du groupe** (cf instructions au point 3)
 - expliquer brièvement votre organisation du travail tout au long du semestre
 - expliquer le/les choix que vous avez dû faire pour implémenter les fonctions `distribution_cartes`, et `complete_et_score`. **Pas de code dans le rapport** juste une explication.
 - expliquer les extensions que vous avez choisies de réaliser : en quoi elles consistent, comment l'on s'en sert, quels choix d'implémentation vous avez dû faire
 - expliquer ce que contiennent vos programmes principaux et comment s'en servir
 - être correctement structuré
 - comporter une bibliographie si vous avez consulté des ressources externes pendant l'élaboration de votre projet
 - utiliser les commandes Latex adéquates (tables des matières, sections, référence et label, listes, ...)
3. un dépôt sur gitlab.isima.fr¹ qui contiendra les différentes versions de votre projet (programmation en Python et rapport en Latex) tout au long du semestre. Veillez à :
 - **nommer votre projet RobotMaster-MiX-BinXX**, en remplaçant les **X** par votre groupe **MI**, puis **votre numéro de binôme**
 - ce que les différents membres du binôme contribuent significativement à alimenter le dépôt
 - faire des `commit` de manière régulière (3 ou 4 par séance au moins, 30 au total grand minimum !), dès lors qu'une fonctionnalité est ajoutée, avec des messages de commit pertinents
 - ce que **votre chargé de TP ait le rôle de Reporter sur votre dépôt** pour pouvoir cloner votre dépôt lorsqu'il/elle vous évaluera.
 - ce que votre dépôt ne contienne pas de fichier auxiliaire inutile (cf utilisation d'un fichier `.gitignore` vue au Semestre 1).

¹Voir Annexe A pour les instructions de configuration.

Notez bien que l'utilisation simultanée de LaTeX et Git a pour conséquence logique l'interdiction d'utiliser un éditeur de LaTeX en ligne, de type Overleaf.

En plus des volets techniques, **il y aura une soutenance orale la semaine du 27 avril 2026**. Plus de détails sur cet oral vous seront donnés ultérieurement.

S'il devait y avoir des modifications de consignes elles pourront vous être données au fur et à mesure du semestre dans les [Annonces Moodle de l'UE Projet](#). Le sujet final du projet sera donc constitué du présent document **et** des consignes qui figureront sur la page web ci-dessus.

Mise en binôme Allez vous inscrire en binôme sur l'activité Choix binôme pour le projet sur [l'espace Moodle de l'UE Projet](#). Les deux membres d'un binôme doivent être dans le même groupe de TP.

Rendu Le rendu final de votre programme est attendu pour le **Vendredi 3 avril à 23h59 au plus tard**. À cette date-là, vous devez :

- **sur Moodle** : avoir déposé votre rapport en PDF (un par binôme est suffisant).
- **Sur Gitlab** avoir donné les accès de votre dépôt gitlab à votre chargé de TP. Le repo doit contenir tous les fichiers '.py' hiérarchisés comme dans le dossier fourni, ainsi que les fichiers sources de votre rapport '.tex' et d'éventuels fichiers auxiliaires nécessaires à leur bonne exécution. **Tout commit passé la date limite sera ignoré ! Quel que soit vos problèmes de connection à 23h58.**

Attention, ce que vous rendez doit refléter votre propre travail. D'une part, les deux membres du binôme doivent travailler équitablement sur le projet, en tout cas sur la partie guidée, le rapport et Git. Pour les extensions, chaque membre du binôme peut décider de partir sur sa propre piste de travail et dans ce cas, seulement la note de la partie Extensions sera différenciée entre les deux individus. D'autre part, il est évidemment hors de question de copier les réponses d'un autre binôme. Le projet que vous rendez va être long, parsemé de commentaires et d'extensions personnalisées: l'équipe pédagogique qui encadre ce cours a des années d'expérience pour détecter la fraude sur les projets rendus... Par ailleurs, la soutenance orale nous permettra de nous rendre compte si un des deux membres du binôme ne maîtrise pas ce qui est écrit dans le rendu.

Chaque membre doit contribuer de manière significative sur les trois aspects du projet, à savoir le code Python, le rapport LaTeX et les commits. Par exemple, il est hors de question qu'un membre d'un binôme travaille uniquement sur le code pendant que l'autre rédige le rapport. Chaque membre du binôme doit toucher à tout.

Vous pouvez bien sûr poser des questions concernant les modalités d'évaluation à votre enseignant durant l'élaboration de votre projet, si des détails ne sont pas clairs. Nous aurons au moins une séance hebdomadaire d'1h30 de TP pour que vous puissiez avancer sur votre projet et poser des questions. **Il est indispensable de compléter par du travail personnel en-dehors de ces séances pour rendre un projet de qualité.**

2 Le thème du projet : RobotMaster

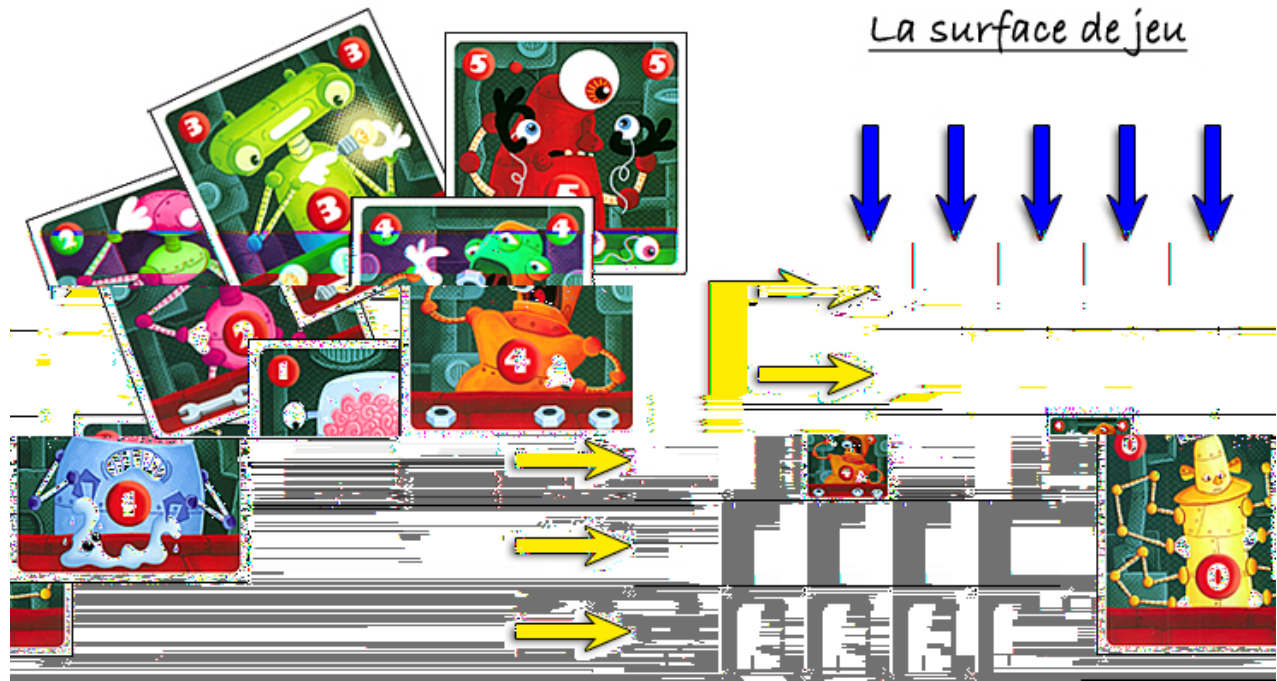


Figure 1: Plateau de jeux initial

Dans ce projet, nous souhaitons programmer le jeu de société **RobotMaster**. Le jeu se joue soit en 1v1, soit par équipe en 2v2. Nous programmeront le jeu en 1v1, une joueuse contre un joueur, mais la plus part des fonctions codées permettront de développer sans trop de changements une version à 4.

Dans ce jeu, la joueuse et le joueur jouent alternativement une carte de valeur 0 à 5 dans une grille de 5 cases par 5 cases. Leur l'objectif est de maximiser le score des colonnes pour la joueuse, et des lignes pour le joueur. Une fois le plateau rempli, chaque ligne (ou colonne) se voit attribué un score, et la joueuse (ou le joueur) avec la ligne de plus petit score est perdant. L'objectif est donc d'assurer un score le plus élevé possible à chacune de ses colonne pour la joueuse (en bleu sur Fig. 1) ou à chacune de ses lignes pour le joueur (en jaune sur Fig. 1).

2.1 Règles du jeu

Cartes et distribution Dans la version que nous implémenterons, les cartes sont numérotées de 0 à 5, avec 6 cartes de chaque exemplaire (soit 36 cartes au total). Le joueur et la joueuse en reçoivent 12 chacune aléatoirement (ces cartes constituent leur main), et une 25ème carte est placée au centre du plateau.

Tour de jeu À tour de rôle, les joueuses placent une carte de leur main sur le plateau. La carte ne peut être posée que sur une case vide qui est adjacente (haut bas gauche droite, et non diagonale) à une case occupée.

Fin et score Quand la grille est complétée, on attribue un score à chaque ligne et colonne comme suit :

- une carte présente en un unique exemplaire rapporte sa valeur au total de la ligne (donc 0, 1, 2, 3, 4, ou 5 points).
- une carte présente en deux exemplaires rapporte dix fois sa valeur au total de la ligne (donc 0, 10, 20, 30, 40, ou 50 points).
- une carte présente en trois exemplaires ou plus rapporte 100 points à la ligne quelque soit sa valeur.

Le score de la joueuse est le score de la plus petite colonne (en bleu sur le Fig. 1), le score du joueur est le score de la plus petite ligne (en jaune sur le Fig. 1). La joueuse (ou le joueur) avec le plus grand score gagne la partie.

3 Implémentations - Informations générales

3.1 Les variables principales

Le code complet fait usages de nombreuses variables, certaines sont omniprésentes et détaillées ci-dessous.

plateau Le plateau est une représentation de l'état du plateau de jeu. C'est une liste de liste d'entiers. Les cases qui ne sont pas encore jouées sont représentées par la valeur 'None'. Par exemple : `[[None, None, 1, 1, 0], [None, 2, None, 3, None], [4, None, None, None, None], [None, 2, None, None, 0], [4, 4, 4, 0, 0]]` est la représentation d'un plateau qui sera affiché dans le terminal de la manière suivante :

	0	1	2	3	4
(0, _)			1	1	0
(1, _)		2		3	
(2, _)	4				
(3, _)		2			0
(4, _)	4	4	4	0	0

Figure 2: Exemple d'affichage de plateau

La gestion du plateau, son affichage sera géré dans le fichier `a_plateau.py`

dictionnaire des joueuses C'est un dictionnaire qui enregistre le nom des joueuses, leur mode de jeu (IA aléatoire, jeu manuel via le terminal etc.), et l'état actuel de leur main (sous la forme de dictionnaire également).

Par exemple une partie opposant Alice, joueuse en manuel avec comme cartes en main : `[0, 0, 1, 1, 2, 4, 4, 4]` et Bob, joueur IA "random" avec comme carte en main : `[0, 0, 0, 2, 2, 3, 5, 5]`. Le **dictionnaire des joueuses** sera : `{0: ["Alice", "m", {0: 2, 1: 2, 2: 1, 3: 0, 4: 3, 5: 0}], 1: ["Bob", "r", {0: 3, 1: 0, 2: 2, 3: 1, 4: 0, 5: 2}]}`. Où Alice est la joueuse "0" et Bob le joueur "1".

dictionnaire des options C'est une manière concise de représenter tous les différents paramétrages du jeux. Pour une partie standard, ce dictionnaire peut prendre la valeur :

`{ 'taille' : 5, 'maxC' : 5, 'nbC' : 6, 'v' : True, 'cartes_distrib' : 12, 'nbJ' : 2 }`. Où *taille* est un entier impair, représentant la taille du plateau. *maxC* représente la valeur maximale que peut prendre une carte, *nbC* le nombre de cartes de chaque valeur, *v* est un boolean qui si Vrai fait que certaines fonctions font plus d'affichages dans le terminal, *cartes_distrib* est le nombre de cartes distribuées à chaque joueuse, et enfin *nbJ* est le nombre de joueuses. Nous partons du principe que ces valeurs satisfont les conditions $(maxC + 1) * nbC \geq cartes_distrib * nbJ + 1 \geq taille^2$. Afin qu'il y ait assez de cartes pour être distribuées aux joueuses (plus une placée au centre du plateau), et que les joueuses puissent remplir le plateau.

(posL, posC) Ce sont des entiers qui font référence à l'emplacement d'une case dans un *plateau*. *posL* indique le numéro de la ligne, *posC* de la colonne.

carte C'est une variable représentant une carte. Cette variable est tout simplement un int.

3.2 Les types utilisés

En fonction des usages, certains types sont plus indiqués que d'autres. Pour être modifié facilement, il est plus simple que *plateau* soit une liste de listes, mais pour être affiché, il sera transformé en chaîne de caractères (voir dernière fonction de `a_plateau.py`). De la même manière, les mains des joueuses sont parfois des listes parfois des dictionnaires notant la fréquence de chaque cartes. Des fonctions permettront de passer d'une représentation à l'autre.

3.3 Les tests

Chaque fonction de la partie guidée est équipée de tests pour que vous puissiez vérifier vous même son bon fonctionnement. **Attention, ce n'est pas parce que votre fonction fonctionne correctement sur un exemple donné qu'elle est correcte dans tous les cas.** C'est cependant un assez bon indicateur et vous permettra de vérifier que vous êtes sur la bonne voie.

Ces tests ont un fonctionnement assez proche de ce que vous avez l'habitude de voir sur Caseine. Cette fois ils sont présents en local sur votre machine. Libre à vous de les regarder, d'en ajouter etc. Bien évidemment, pour l'évaluation, ce ne sera pas sur votre fichier modifiable (mais bien celui des enseignants) que vos fonctions seront validées.

Pour effectuer les tests, il faut tout d'abord que le module [pytest](#) soit installé sur votre machine. Vous pouvez ensuite lancer les tests (par exemple du fichier `a_test.py`) en tapant dans le terminal

```
$ pytest a_test.py
```

Nous recommandons l'usage de l'option `-v` qui donnera plus de détails sur les tests effectués. Par exemple (en se plaçant dans le terminal en amont du dossier `partie_guidée`), la commande suivante exécute tous les tests du sous dossier `partie_guidée` et pour chaque fonction dit si le test était positif ou négatif.

```
$ pytest partie_guidée -v
```

```
alvigny@LT-SCG3284NJQ: ~/git/Teaching/25-26/Projet-L1/ProjetL1-RobotMaster/Correction$ pytest partie_guidée -v
===== test session starts =====
platform linux -- Python 3.10.12, pytest-8.3.5, pluggy-1.5.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/alvigny/git/Teaching/25-26/Projet-L1/ProjetL1-RobotMaster/Correction
collected 42 items

partie_guidée/a_test.py::test_creer_plateau_1 PASSED [ 2%]
partie_guidée/a_test.py::test_creer_plateau_2 PASSED [ 4%]
partie_guidée/a_test.py::test_creer_plateau_3 PASSED [ 7%]
partie_guidée/a_test.py::test_creer_plateau_4 PASSED [ 9%]
partie_guidée/a_test.py::test_cases_libres_1 PASSED [11%]
partie_guidée/a_test.py::test_cases_libres_2 PASSED [14%]
partie_guidée/a_test.py::test_plateau_to_string_1 PASSED [16%]
partie_guidée/a_test.py::test_plateau_to_string_2 PASSED [19%]
partie_guidée/a_test.py::test_plateau_to_string_3 PASSED [21%]
partie_guidée/b_test.py::test_new_pile_cartes_1 PASSED [23%]
partie_guidée/b_test.py::test_distribution_cartes_1 PASSED [26%]
partie_guidée/b_test.py::test_distribution_cartes_2 PASSED [28%]
partie_guidée/b_test.py::test_distribution_cartes_3 PASSED [30%]
partie_guidée/b_test.py::test_liste_to_dico_1 PASSED [33%]
partie_guidée/b_test.py::test_liste_to_dico_2 PASSED [35%]
partie_guidée/b_test.py::test_cases_voisines_1 PASSED [38%]
partie_guidée/b_test.py::test_cases_voisines_2 PASSED [40%]
partie_guidée/b_test.py::test_cases_voisines_3 PASSED [42%]
partie_guidée/b_test.py::test_cases_voisines_4 PASSED [45%]
partie_guidée/b_test.py::test_emplacement_jouable_1 PASSED [47%]
partie_guidée/b_test.py::test_emplacement_jouable_2 PASSED [50%]
partie_guidée/b_test.py::test_place_carte_1 PASSED [52%]
partie_guidée/b_test.py::test_place_carte_2 PASSED [54%]
partie_guidée/c_test.py::test_init_tuple_joueurs_1 PASSED [56%]
partie_guidée/c_test.py::test_init_tuple_joueurs_2 PASSED [59%]
partie_guidée/c_test.py::test_configuration_textuel PASSED [61%]
partie_guidée/c_test.py::test_choix_carte_random PASSED [64%]
partie_guidée/c_test.py::test_choix_carte_manuel_1 bienveillant PASSED [66%]
partie_guidée/c_test.py::test_choix_carte_manuel_2 mauvaise entree PASSED [68%]
partie_guidée/c_test.py::test_test_choix_at_pose_carte_1 PASSED [71%]
partie_guidée/c_test.py::test_test_choix_at_pose_carte_2 PASSED [73%]
partie_guidée/d_test.py::test_ligne_to_dico_1 PASSED [75%]
partie_guidée/d_test.py::test_ligne_to_dico_2 PASSED [78%]
partie_guidée/d_test.py::test_score_ligne_1 PASSED [80%]
partie_guidée/d_test.py::test_score_ligne_2 PASSED [83%]
partie_guidée/d_test.py::test_score_joueuse_1 PASSED [85%]
partie_guidée/d_test.py::test_score_joueuse_2 PASSED [88%]
partie_guidée/e_test.py::test_test_tour PASSED [90%]
partie_guidée/f_test.py::test_jeu_random_ensemble PASSED [93%]

===== 42 passed in 0.04s =====
alvigny@LT-SCG3284NJQ: ~/git/Teaching/25-26/Projet-L1/ProjetL1-RobotMaster/Correction$
```


4 Implémentations - Partie Guidée

Le projet a été découpé en de nombreuses fonctions, on demande de respecter cette structure. Les fonctions demandées doivent **IMPERATIVEMENT** respecter les spécifications qui sont données dans le sujet. Respectez **STRICTEMENT** les affichages demandés : à la virgule près, à l'espace près, bref au caractère près. Vous avez le droit d'écrire des fonctions auxiliaires non explicitement demandées uniquement si elles servent à implémenter les fonctions demandées.

Autant que possible, **testez toutes vos fonctions au fur et à mesure que vous les écrivez**. Avant de coder une fonction, vous devez être convaincus que toutes les fonctions précédentes fonctionnent.

Commencez par [télécharger sur Moodle](#) le squelette `Squelette_RobotMaster.zip` du projet fourni. Il faudra le compléter pour terminer la partie guidée. En particulier, le code a été divisé en plusieurs fichiers.

Votre programme devra être clair et commenté. **Il est obligatoire de commenter votre programme pour des fonctions avec plus de dix lignes de code**. Privilégiez les commentaires au niveau des boucles (for, while) et des conditionnelles (if) pour montrer à votre intervenant vos choix d'implémentation.

Votre programme doit être régulièrement mis à jour dans votre repository GitLab. Rappel : les deux membres d'un binôme doivent chacun "commiter" de manière significative.

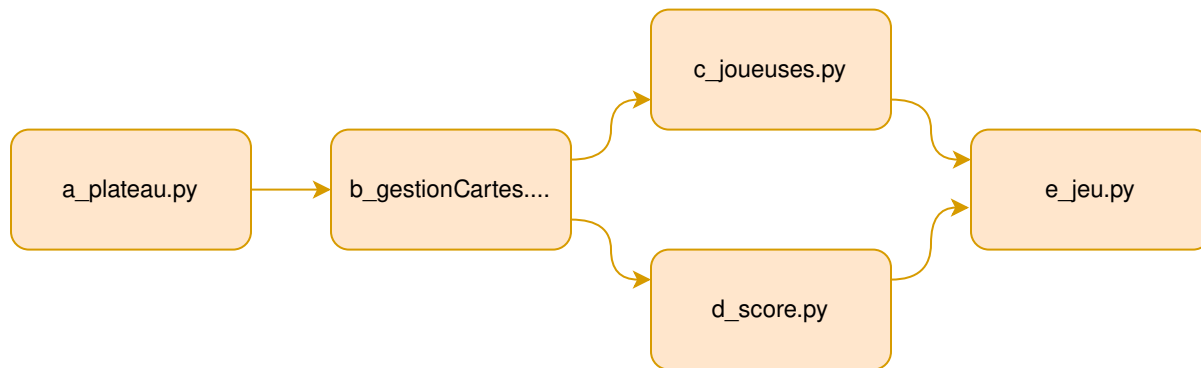


Figure 3: Les fichiers de la Partie Guidée à remplir et leurs dépendances.

4.a Fichier `a_plateau.py`

L'objectif de cette section est de créer et afficher un plateau.

4.a.1 Fonction `creer_plateau`

Écrire une fonction `creer_plateau` qui prend un argument optionnel `taille` qui est un entier positif, initialisé par défaut à 5. Si `taille` est pair ou négatif, on envoie un message d'erreur. Sinon, on crée une matrice de dimension `taille × taille` remplie de `None`. **Votre code ne doit pas contenir la valeur numérique 5 !!**

4.a.2 Fonction `afficher_coordonnees` (fournie)

La fonction `afficherCoordonnees` est déjà implémentée. Elle ne sera pas utilisée dans les fonctions qui suivent. Son objectif est simplement de vous permettre de comprendre quelles coordonnées sont associées à telle ou telle case.

4.a.3 Fonction `cases_libres`

La fonction `cases_libres` prend en argument un plateau et retourne la liste des cases vides sous la forme de tuple (abscisse ordonnée). Donc dans l'exemple de la Fig. 1 : [(0, 0), (0, 1), (1, 0), (1, 2), (1, 4), (2, 1), (2, 2), (2, 3), (2, 4), (3, 0), (3, 2), (3, 3)]. Peu importe l'ordre des éléments de cette liste.

4.a.4 Fonction plateau_to_string

Cette fonction est la plus délicate de cette section. Elle retourne **AU CARACTÈRE PRÈS** une chaîne de caractères décrite dans les exemples ci dessous. Elle prend en argument un plateau et, éventuellement une chaîne de 3 caractères représentant une case vide :

	0	1	2	3	4
(0,_)			1	1	0
(1,_)		2		3	
(2,_)	4				
(3,_)		2			0
(4,_)	4	4	4	0	0

Ou bien par exemple

	0	1	2
(0,_)			3
(1,_)			2
(2,_)	1		

Cahier des charges :

- Chaque case contient 3 caractères (3 espaces, ou un chiffre entre deux espaces),
- Il y a trois espaces entre (0,_) et le premier trait vertical,
- Les indices colonnes doivent être alignés avec le caractère central de chaque case,
- Les lignes horizontales tout en haut et en bas du plateau doivent se terminer au niveau du dernier trait vertical du plateau, quelque soit la taille du plateau.
- Il y a deux espaces après le dernier chiffre de la deuxième ligne pour que chaque ligne fasse le même nombre de caractères.

Rappel du S1 : la commande `\n` peut être utilisée dans une chaîne de caractères pour sauter une ligne.

4.b Fichier b_gestionCartes.py

Cette section consiste à programmer les fonctions qui vont gérer cartes, les créer, les distribuer, et les placer sur le plateau.

4.b.1 Fonction new_pile_cartes

La fonction `new_pile_cartes` crée une liste mélangée de cartes. La valeur la plus haute et le nombre de cartes par valeur sont renseignés dans *dictionnaire des options*, les cartes vont de 0 à *maxC* incluse (5 par défaut). Il y a *nbC* (par défaut 6) cartes de chaque valeurs.

Par exemple `new_pile_cartes({'maxC':3, 'nbC':2})` peut retourner `[1,0,2,1,2,3,3,0]`.

4.b.2 Fonction distribution_cartes

La fonction `distribution_cartes` crée une list de *nbJ* + 1 éléments. Le premier élément est une carte (qui sera jouée au milieu), suivit *nbJ* listes, chacun de *cartes_distrib* cartes, représentant la main de chaque joueuse. Toute ces listes sont remplis en prenant les cartes de l'entrée `pile_carte`.

Par exemple la fonction `distribution_cartes([0,1,2,3,4,5,6],{'nbJ' : 2, 'cartes_distrib':3})` peut retourner la liste `[3,[1,2,6],[4,5,0]]`.

4.b.3 Fonction liste_to_dico

La fonction `liste_to_dico` transforme une liste de cartes en un dictionnaire. Les clés du dictionnaire sont les valeurs de cartes possibles (de 0 à *maxC*). Et les valeurs sont le nombre de cartes correspondantes.

Par exemple pour la liste `[2,0,0]` et *maxC* = 2, la fonction retourne `{0:2, 1:0, 2:1}`

4.b.4 Fonction `cases_voisines`

La fonction `cases_voisines` prend en argument un *plateau* et les coordonnées d'une case *posL*, *posC*. Elle renvoie la liste des cases qui sont voisines de la case donnée en entrée. La liste renvoyée est donc de taille au plus 4.

Attention à bien gérer les entrées qui ne sont pas des coordonnées de case du plateau, ainsi que les cas où la case est située sur un bord (ou deux).

4.b.5 Fonction `emplacement_jouable`

La fonction `emplacement_jouable` prend en argument un *plateau* et les coordonnées d'une case *posL*, *posC*. Elle retourne un boolean `True` si il est possible de jouer sur cette case (`False` sinon). La case doit être vide et doit avoir au moins une voisine non vide.

4.b.6 Fonction `place_carte`

La fonction `place_carte` prend en argument un *plateau*, des coordonnées *posL*, *posC* et une *carte*. Elle ne retourne rien mais modifie le *plateau* pour ajouter la *carte* en position (*posL*,*posC*) si la case est jouable.

4.c Fichier `c_joueuses.py`

Ce fichier traite de la création des joueuses, de leur paramétrisation (nom, mode de jeu, etc.), ainsi que leur placement des cartes. De nombreuses fonctions utilisent régulièrement les inputs de la console. Il est important d'utiliser les blocs `try et except` : pour que le programme ne crash pas si l'entrée n'est pas bien formatée (une lettre à la place d'un chiffre par exemple).

4.c.1 Fonction `init_tuple_joueuses`

La fonction `init_tuple_joueuses` retourne un tuple de nom pour les joueuses. Par défaut "Alice" et "Bob". Mais si l'option *v* est `True` dans *dictionnaire des options*, on demandera alors dans la console de rentrer le nom de la joueuse et du joueur.

4.c.2 Fonction `configuration_textuel`

La fonction `configuration_textuel` prend en entrée un tuple de nom pour les joueuses. Elle demande dans la console quel mode de jeu est souhaité pour chacune : *m* pour manuel, et *r* pour random. La fonction crée le *dictionnaire des joueuses* où les clés sont 0 et 1, et les valeurs un tuple nom, mode de jeu, dictionnaire de la main (initialisé à vide). Un retour possible de la fonction est : `{0: ["Alice", "m", {}], 1: ["Bob", "r", {}]}`

4.c.3 Fonction `choix_carte_manuel`

La fonction `choix_carte_manuel` demande à la joueuse dont le *nom* est en entrée de choisir une carte de sa *main*. On affichera son *nom*, sa *main* et le *plateau*.

La fonction doit retourner un tuple (*carte*, *posL*, *posC*) jouable, redemandant à la joueuse de nouvelles entrées tant que ce n'est pas le cas.

4.c.4 Fonction `choix_carte_random`

La fonction `choix_carte_random` retourne un tuple (*carte*, *posL*, *posC*) choisit aléatoirement parmi les cartes de la *main* et les positions jouables. Les choix possibles sont déterminés grâce aux informations de l'entrée *dico_main* et *dictionnaire des options*. On vérifie bien que ce coup est jouable sur le *plateau* avant de retourner un tuple.

4.c.5 Fonction choix_et_pose_carte

La fonction `choix_et_pose_carte` effectue le tour de la `joueuse_active` (un int égal à 0 ou 1). Elle appelle la fonction `choix_carte_manuel` ou `choix_carte_random` en fonction des informations dans *dictionnaire des joueuses*, et la fonction `place_carte` du fichier `b_gestionCartes.py`, avant de retirer la `carte` du la `main` de la joueuse. Enfin, si dans *dictionnaire des options* la valeur de '`v`' est `True`, on affiche un message comme '*A pose la carte x sur la case i,j* ' en remplaçant `Axij` bien évidemment.

4.d Fichier `d_score.py`

Ce fichier a pour but de calculer les scores des joueuses.

4.d.1 Fonction `init_dico_cartes`

La fonction `init_dico_cartes` crée un dictionnaire dont les clés sont des int correspondant aux valeurs possibles des cartes, et les valeurs sont initialisées à 0. Dans *dictionnaire des options*, le champ `maxC` donne la valeur maximale qu'une carte peut avoir.

4.d.2 Fonction `colonne_to_dico`

La fonction `colonne_to_dico` prend un `plateau`, deux entiers `joueuse_active` et `i`, avec *dictionnaire des options* et retourne un dictionnaire. Si `joueuse_active` est impaire, on regarde la `i` ème ligne du `plateau` (sinon la `i` ème colonne). Cette ligne ou colonne est transformée en dictionnaire comme le fait la fonction `liste_to_dico`.

4.d.3 Fonction `score_ligne`

Étant donné une ligne (ou une colonne) sous la forme d'un dictionnaire qui recense le nombre d'occurrences de chaque carte, la fonction `score_ligne` calcule le score de la ligne en accord avec les règles du jeu Robot Master.

4.d.4 Fonction `score_joueuse`

La fonction `score_joueuse` retourne un tuple d'entiers. Si `joueuse_active` est paire, on regarde le score des colonnes (de la joueuse en bleue dans Fig. 1), sinon le score des lignes (du joueur en jaune dans Fig. 1). On retourne un tuple contenant le score ainsi que l'indice de la colonne (ou la ligne) qui réalise ce score.

4.d.5 Fonction `victoire`

La fonction `victoire` retourne un tuple d'entiers contenant : le score de la joueuse, l'indice de la colonne correspondante, le score du joueur, l'indice de la ligne correspondante.

4.e Fichier `e_jeu.py`

Ce fichier gère un tour de jeu complet et permet de lancer une partie.

4.e.1 Fonction `tour`

La fonction `tour` effectue un tour de jeu de la `joueuse_active`. si l'option '`v`':`True` est présente dans *dictionnaire des options*, on affichera un message comme "au tour de ..." en début et "fin du tour" en fin de tour.

4.e.2 Fonction `jeux`

La fonction `jeux` exécute le jeu. Elle génère le `plateau`, les dictionnaires *dictionnaire des joueuses* et *dictionnaire des options*, distribue les cartes et fait jouer les `joueuses` en alternance jusqu'à ce que le `plateau` soit plein. Enfin on affiche le résultats (nom de la gagnante, et les scores).

5 Implémentations - Stratégie et IA

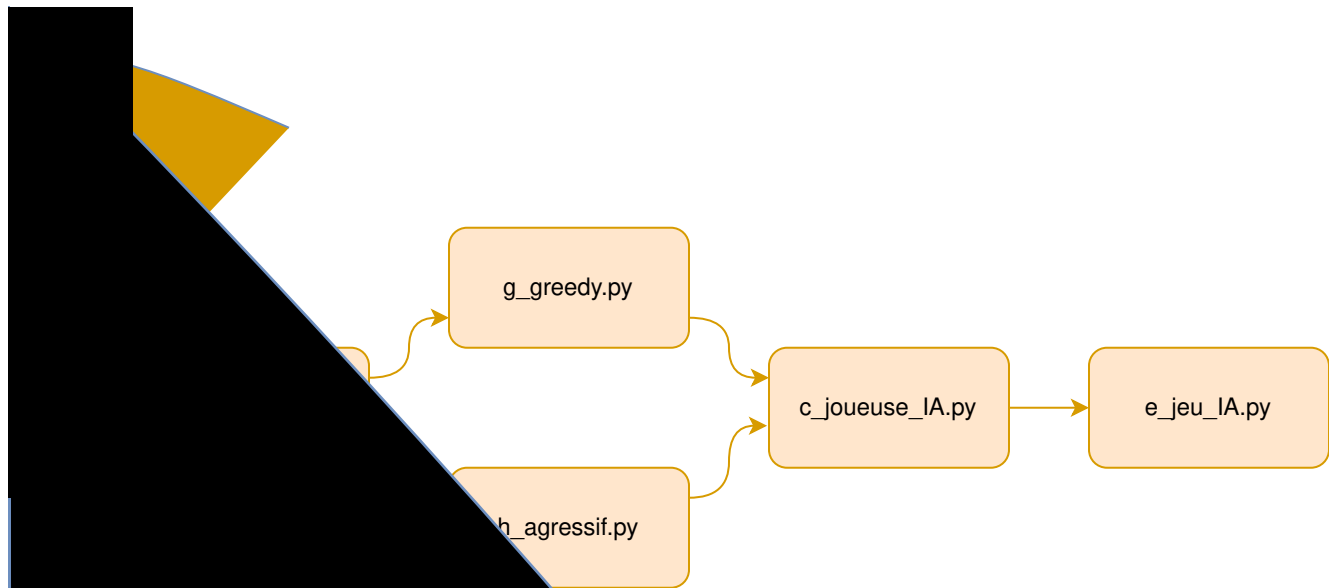


Figure 4: Les fichiers de la partie IA à remplir et leur dépendances.

5.c Fichier c_joueuses_IA.py

Légère modification du fichier `c_joueuses.py` de la partie guidée. On importe des fichiers `g_greedy.py` et `h_agressif.py` (et éventuellement d'autres en fonction de vos extensions). On modifie les fonctions `configuration_textuel` et `choix_et_pause_carte` pour intégrer les différentes stratégies développées: g: greedy, a: agressif.

5.e Fichier e_jeu_IA.py

Légère modification du fichier `e_jeu.py` de la partie guidée. On importe toujours les fonctions du fichiers `d_score.py` de la partie guidée, et les fonctions du nouveau fichier `c_joueuses_IA.py`. Il n'y a sinon pas de changement des fonctions `tour` et `jeux`.

5.f Fichier f_fonctions_additionnelles.py

Contient des fonctions supplémentaires facultatives, mais qui peuvent être utiles pour les stratégies ci-dessous. Il y a néanmoins une fonction obligatoire dans ce fichier.

5.f.1 Fonction `score_complet_joueuse`

Au lieu de simplement retourner le score de la colonne la plus petite de la `joueuse_active` cette fonction retourne la liste des scores des lignes, en ordre croissant. Cette fonction donne un meilleur aperçu de la situation, notamment en début de partie, quand 3 ou 4 colonnes valent 0 point, avoir un meilleur score sur les autres colonnes est significatif.

5.g Fichier g_greedy.py

Ne contient qu'une seule fonction `choix_carte_greedy`, qui prend un `plateau`, la main de la joueuse (sous la forme d'un dictionnaire), ainsi que le `dictionnaire des options` et l'entier `joueuse_active`. Cette fonction retourne un tuple (`carte`, `posL`, `posC`) qui maximise le `score_complet` de la `joueuses_active` parmi tout les coups jouables.

5.h Fichier `h_agressif.py`

On va chercher, étant donnée une ligne partiellement remplie, à calculer toutes les manières de la compléter. On pourra alors affecter à une ligne un `score_max` (quoi qu’il arrive la ligne ne fera pas plus) et un `score_min` (la ligne ne peut pas faire moins). Enfin on définira la stratégie `choix_carte_agressif` qui minimise le score maximal que peut obtenir l’adversaire.

5.h.1 Les fonctions `tous_les_scores_possibles` et `complete_et_score`

La fonction `tous_les_scores_possibles` prend en argument un `dico_ligne`, un dictionnaire des cartes restantes (si tous les 5 sont déjà joués, on sait qu’aucun 5 ne peut compléter la ligne étudiée), et le *dictionnaire des options*. À l’aide de la fonction `complete_et_score`, elle calcule toutes les complétions possibles et retourne la liste de tout les scores possibles pour cette ligne.

La fonction `complete_et_score` est une fonction récursive. Si la ligne en argument est “complete”, on se contentera d’ajouter son score à la liste des `score_possibles`. Sinon on parcourt toutes les cartes restantes (présentes en au moins 1 exemplaire dans `dico_cartes_restantes`); et pour chaque carte on calculera toutes les complétions possibles si cette carte est ajoutée à la ligne et retirée du dictionnaire des cartes restantes.

5.h.2 La fonction `score_max_potentiel_complet_joueuse`

Cette fonction fait appel à la fonction `tous_les_scores_possibles` pour calculer le score complet maximal que peut atteindre une joueuse (pour chaque ligne, quel est le score de la meilleure complétion).

5.h.3 La fonction `choix_carte_agressif`

Cette fonction joue la carte qui minimisera le score maximal potentiel de la joueuse adverse. Pour chaque coup possible on calcul le `score_max_potentiel_complet_joueuse` de l’adversaire, et on prend le coup qui minimise ce résultat.

5.i Fichier `i_meilleurIA.py` (extension battez les profs)

Ce fichier est une extension possible de votre projet. Vous y ajouterez une fonction `choix_carte_IA` (ainsi que toutes les fonctions nécessaires à son exécution).

Si vous faites cette extension, vous modifierez également le fichier `c_joueuses_IA.py` pour accepter les configurations `i` et `p`. Les profs développeront une stratégie `choix_carte_prof` et nous comparerons l’efficacité des IA.

Plus d’informations sur cette extension vous seront communiquées pendant le semestre.

6 Partie Extensions

En plus des fonctionnalités présentées dans la Partie Guidée et la partie IA, vous devez réaliser d'autres fonctionnalités, peu ou pas guidées, que l'on appellera des **extensions**. Le but est de personnaliser votre projet pour le différencier des autres binômes, et aussi de montrer que vous êtes capables de concevoir une solution même lorsque vous êtes moins guidés. Pour cette partie, vous serez probablement amenés à écrire plusieurs fonctions auxiliaires qu'il vous faudra définir vous mêmes. A vous de structurer votre travail avec des fonctions adéquates.

Toutes ces extensions doivent être **uniquement dans le dossier “extensions”**. Vous pouvez copié-collé depuis votre partie guidée, **mais ne modifiez pas vos partie guidée et IA**.

L'idée générale de la [Partie Extensions](#) est : imaginez d'autres fonctionnalités (ou améliorations de fonctionnalités existantes) et implémentez-les. Faites preuve d'originalité. Néanmoins, nous vous proposons quelques idées ci dessous pour le cas où vous seriez en manque d'imagination. Si vous n'êtes pas très inspirés, vous pouvez par exemple combiner deux extensions issues des propositions, avec une que vous aurez inventée. Au contraire, si vous avez plein d'idées, vous pouvez proposer trois extensions différentes de celles présentées ci-dessous. L'important est de proposer un contenu cohérent et innovant.

Nous ne pouvons pas répondre à la question : *combien d'extension faut-il réaliser ?* cela dépend de la complexité et de la qualité des extensions implémentées. Vous pouvez demander des précisions pendant les TP.

Vous devrez, dans votre rapport, expliquer quelles extensions vous avez réalisé, comment s'en servir, et brièvement comment vous avez procédé. **Très important** : quand vous rendrez votre projet, vous laisserez à votre intervenant un programme principal de vos extensions avec du code très bien commenté permettant d'expliquer et d'illustrer (avec des exemples à décommenter) les nouvelles fonctionnalités que vous venez d'ajouter. Commentez tous les autres tests. Vous devez faire l'effort de guider votre intervenant à prendre en main vos extensions : nous ne connaissons absolument pas les subtilités de votre code. Soyez pédagogues !

6.1 Propositions

4 joueuses Implémentez la version 2v2, chaque joueuse reçoit 7 cartes.

Battez les profs Faites une IA meilleure que la notre. Voir les instructions dans la [Section 5.i](#).

Menu console Vous pouvez améliorer la phase d'initialisation d'une partie de RobotMaster avec un menu interactif pour déterminer les configurations, etc. Mais aussi, par exemple : genrer les joueurs/joueuses, sauvegarder les scores des joueuses en fin de partie, demander s'ils veulent faire une revanche, donner des statistiques de partie (graphique de l'évolution du score ou autre...). Bref, vous pouvez vous faire plaisir ! Ce menu peut être affiché soit dans la console, soit dans une fenêtre graphique en utilisant Tkinter ou un autre package.

Interface graphique Vous pouvez remplacer l'affichage console que nous avons proposé dans la partie guidée par une fenêtre graphique et rendre le jeu plus visuel. Choisissez la bibliothèque graphique de votre choix (Tkinter par exemple). Vous pouvez ajouter des boutons, des menus déroulants, des images pertinentes pour représenter cartes, joueuses, etc. Faites vous plaisir !

6.2 Vos extensions

Vous n'êtes pas obligé de vous conformer à la liste ci-dessus : vous pouvez proposer vos extensions ! Ayez de l'imagination !

A Annexe – Configuration du dépôt GitLab

A.1 Bien démarrer

Dans les salles SCI 002 à SCI 006 : il faut se connecter en cliquant sur *Portail pédagogique VDI* et ensuite Linux. Si vous arrivez sur un écran de veille avec l'heure indiquée en grand au centre, réveillez le système (par la touche Échap ou un clic-glissé vers le haut) et ré-authentifiez-vous.

Dans les salles SCI 007, SCI 008, PHY 214 et ISIMA B011 : normalement, ces salles disposent d'un véritable système d'exploitation Linux installé. Si l'ordinateur est déjà démarré sous Windows, il est nécessaire de redémarrer en guettant attentivement le menu de *dual-boot* (double-démarrage). Ce menu vous permet de choisir entre plusieurs systèmes d'exploitation. En l'absence de choix explicite de l'utilisateur, le système par défaut (probablement Windows) est automatiquement pris au bout d'un certain temps (d'où l'attention requise). Vous devez choisir, à l'aide des flèches du clavier (haut/bas), une distribution (variante) de Linux : Ubuntu, Debian, Fedora, ... puis valider avec Entrée.

Depuis n'importe quelle machine et n'importe quel système d'exploitation : vous avez accès aux machines virtuelles depuis un navigateur web : <https://guacamole.isima.fr/> (ou en installant le logiciel VMWare Horizon). Pour la suite des instructions, voir le paragraphe sur les salles SCI 002 à SCI 006.

Pour optimiser les ressources, n'utilisez pas ces machines virtuelles si vous êtes déjà sur un ordinateur disposant de Linux avec tous les outils nécessaires.

A.2 Créer un projet GitLab

Choisissez un membre de l'équipe – que l'on nommera le *propriétaire* dans la suite – et faites-lui créer un nouveau projet **privé** sur gitlab.isima.fr comme ceci :

1. cliquez sur le bouton bleu Nouveau projet ;
2. choisissez la boîte Create blank project ;
3. choisissez un nom de projet **RobotMaster-MiX-BinXX**, en remplaçant les X par votre groupe MI, puis votre numéro de binôme, puis valider via le bouton bleu Create project.

C'est ce projet qui va contenir vos fichiers .py mais aussi votre rapport LaTeX pour le projet du RobotMaster. Une fois créé de la sorte, le dépôt peut d'ores et déjà être cloné par le propriétaire en une copie de travail sur sa machine.

Dans un second temps, le propriétaire doit ajouter l'autre membre de l'équipe au dépôt **gitlab**. Ceci peut être fait comme suit, depuis la page principale du projet :

1. allez dans le menu *Project information* (panneau latéral de gauche), puis sélectionnez le sous-menu *Membres* ;
2. ajoutez l'autre membre de l'équipe, en précisant le rôle *Maintainer* (pour donner des droits en écriture et en lecture) mais sans préciser de date d'expiration de l'accès.

Désormais, l'autre membre peut également cloner le dépôt sur sa machine.

Enfin, par la même méthode, ajoutez votre chargé(e) de TP au projet, en lui octroyant le rôle **Reporter**.

En cas de problème, contactez au plus vite votre chargé de TP.

Configuration avancée. Il est possible d'aller plus loin dans la configuration de **gitlab**, en utilisant les clés **ssh** (et l'accès **ssh**). Cette configuration n'est pas obligatoire mais très pratique si vous en avez marre de taper vos identifiants à chaque **pull** ou **push**. Elle ne concerne pas directement votre dépôt, mais votre compte personnel sur le **gitlab** de l'ISIMA. Cela se passe [ici](#), où vous trouverez des liens de documentation pour créer votre clé **ssh**. Une fois votre compte **gitlab** configuré correctement, vos identifiants ne seront plus demandés à chaque échanges (**pull**, **push**) avec le dépôt **gitlab**.

A.3 Premier commit

On va commencer par cloner le projet que vous venez de créer sur GitLab, si ce n'est pas déjà fait.

1. Pour cela, retournez sur l'onglet principal de votre projet. Vous allez remarquer un bouton bleu `Clone` en haut à droite: cliquez dessus. Si vous êtes à l'aise avec SSH, configurez votre projet en accès SSH. Sinon, copiez l'URL de la case *Clone with HTTPS*. Si vous voulez vous familiariser plus tard avec SSH, ce sera évidemment possible.
2. Utilisez la commande `git clone` dans le terminal Linux, suivie de l'URL copié précédemment. Veillez à exécuter cette commande dans un dossier prêt à accueillir votre projet (pas dans Téléchargements par exemple). Le terminal vous demande de rentrer vos identifiants GitLab, puis clone votre projet. Vous pouvez remarquer qu'un dossier est apparu, contenant `.git` ainsi qu'un `ReadMe`.

Maintenant, nous allons ajouter du contenu Python à votre projet pour que vous commenciez à coder.

1. Téléchargez le squelette `Squelette_RobotMaster.zip` sur Moodle et ajoutez le dans votre dossier.
2. On va maintenant signifier à Git que l'on a effectué des modifications à notre fichier. Dans le terminal, exécutez `git add .` ou `git add <nom_dossier>`.
3. Lancez `git status`. N'oubliez pas que cette commande vous permet à tout moment de visualiser ce que vous venez d'ajouter à l'index.
4. Faites un commit ! On vous laisse retrouver la syntaxe (cf cours de S1 ou mémo ci-dessous). Vous pourrez ajouter en commentaire "Ajout squelette partie guidée" par exemple.
5. Pour l'instant, vos modifications ont été seulement prises en compte sur votre version locale du projet (c'est-à-dire sur l'ordinateur sur lequel vous êtes). On veut déplacer ses modifications sur le dépôt GitLab de votre projet, afin que votre binôme (et votre encadrant !) puisse les voir. Lancez `git push`. Lorsque c'est terminé, actualisez la page GitLab de votre projet. Le commit doit apparaître.

A.4 Pour la suite

Diverses indications pour la suite du projet:

1. A chaque fois que vous travaillez sur le projet, pensez à `git pull` (ou `git clone`) pour récupérer la dernière version du projet (au cas où votre binôme ait fait des modifications).
2. Testez également lors des deux premières séances un premier commit pour le second membre du binôme, pour vérifier que tout fonctionne.
3. Le dépôt GitLab contiendra également le rapport LaTeX de votre projet. Vous pouvez configurer un `.gitignore` pour éviter de prendre en compte les fichiers de compilations (`.aux`, `.toc`, etc.).

De manière générale, vous devez vous référer aux cours du S1 (Python ou Git+LaTeX) avant d'appeler votre encadrant à l'aide: ils contiennent tout ce dont vous avez besoin, au moins pour la partie guidée du Projet.

A.5 Mémo git

Bien sûr, ce mémo n'est pas exhaustif. D'autres commandes et d'autres options sont disponibles, n'hésitez pas à consulter la documentation (en ligne <https://git-scm.com/doc>, tapez par exemple `git checkout` dans la barre de recherche pour voir les options possibles de `git checkout` ; ou le `man`, tapez par exemple `man git checkout` dans le Terminal ; voir aussi `man giteveryday` pour un rappel des commandes les plus courantes, `man gitglossary`, `man gittutorial`, `man gitworkflows`, ...). Ou plus simplement, utilisez Google pour résoudre votre souci : cela vous enverra sur des forums où des personnes ont le même problème que vous.

Commande et exemple	Description
<code>git init</code>	Initialiser un dépôt vide
<code>git status</code>	Voir l'état de l'index
<code>git add fichier</code> <code>git add tp.py</code>	Ajouter un fichier à l'index
<code>git commit -m message</code> <code>git commit -m "Ajout de telle fonctionnalité"</code>	Faire un commit avec le contenu actuel de l'index
<code>git diff</code>	Visualiser les différences entre votre copie de travail et votre index (qui seraient donc laissées de côté si l'on faisait un commit immédiatement)
<code>git diff --cached</code> ou <code>git diff --staged</code>	Visualiser les modifications qui sont actuellement dans l'index, prêtes pour le prochain commit
<code>git add --patch fichier</code> <code>git add --patch tp.py</code>	Ajouter une partie des modifications d'un fichier à l'index (accès à l'éditeur pour indiquer les modifications à prendre en compte)
<code>git clone url</code> <code>git clone https://gitlab.isima.fr/aulagout/tp-projet</code>	Cloner un dépôt git distant
<code>git pull</code>	Récupérer la dernière version d'un dépôt git distant déjà configuré
<code>git push</code>	"Pousser" les commits faits localement vers un dépôt distant déjà configuré. Seuls les commits de la branche contenant la HEAD seront recopiés sur le dépôt distant.
<code>git branch maBranche</code>	Crée une nouvelle branche à l'endroit où se trouve la HEAD (référence sur le commit où se trouve la HEAD). La HEAD ne se met pas automatiquement sur cette nouvelle branche
<code>git checkout maBranche</code>	Passer dans la branche maBranche : la HEAD se met dans cette branche, et le répertoire de travail prend l'état du dernier commit de cette branche.
<code>git branch maBranche</code> puis <code>git checkout maBranche</code> ou, 2-en-1 sur les versions récentes : <code>git switch --create maBranche</code>	Débuter une nouvelle branche à l'endroit où l'on se trouve, et continuer sur cette branche.
<code>git checkout refDunCommit</code> <code>git checkout HEAD~ (reculer d'un cran)</code> <code>git checkout HEAD~2 (reculer de 2 crans)</code> <code>git checkout 316e0cd5d99084bf6a516a58f6a4eb92db9d5e5d</code>	Le répertoire de travail prend l'état enregistré dans ce commit, la HEAD se détache de la branche en cours pour se placer sur ce commit. Si l'on souhaite repartir de ce commit pour enchaîner sur un autre commit, il faut créer une branche ici.
<code>git merge autreBranche</code>	Fusionne la branche autreBranche dans la branche actuelle
<code>git reset --soft refDunCommit</code>	Place la HEAD sur le commit indiqué. Le répertoire de travail n'est pas modifié. L'index n'est pas remis à zéro.
<code>git reset</code>	Remet l'index à zéro. Le répertoire de travail n'est pas modifié.
<code>git reset --hard refDunCommit</code>	Remettre tout à l'état du commit indiqué : la HEAD (et la branche courante, si l'on est sur une branche) est déplacée sur ce commit, l'index est remis à zéro, et le répertoire de travail est mis à l'état enregistré dans le commit . <i>Perte de données si l'état du répertoire de travail précédent le reset n'a pas été commité</i>